

Homework 1

Student Name: Lyon Kee

1. Understand Zipf's Law

```
1.1. import re
    import string
    import os

    yelp_restaurants = []
    for yelp_fp in os.listdir('./yelp/'):
        with open('./yelp/' + yelp_fp, "r") as restaurant:
            yelp_restaurants.append(json.load(restaurant))

    TTF = {}
    DF = {}
    stop_words = set(nltk.corpus.stopwords.words('english'))

    for restaurant in yelp_restaurants:
        for review in restaurant['Reviews']:
            tokens = re.sub(r"\b\d+\b", "#", review['Content']
                           .translate(str.maketrans('', '', string.punctuation))
                           ).lower()

            # Tokenize by word
            tokens = nltk.word_tokenize(tokens)

            # # Stopword removal
            tokens = [word for word in tokens if not word in stop_words]

            # Token stemming
            tokens = [nltk.stem.PorterStemmer().stem(token) for token in tokens]

            # Write back to the object
            # review['Content'] = tokens

            for token in tokens:
                TTF[token] = TTF.get(token, 0) + 1

            for token in set(tokens):
                DF[token] = DF.get(token, 0) + 1

    # print(json.dumps(yelp, sort_keys=True, indent=3))
```

```

#sort TTF
TTF = {key: val
        for key, val in
            sorted(TTF.items(), key=lambda pair:pair[1], reverse=True)}
print('TTF: ', TTF)
#sort DF
DF = {key: val
        for key, val in
            sorted(DF.items(), key=lambda pair:pair[1], reverse=True)}
print('DF: ', DF)

import matplotlib.pyplot as plt

plt.yscale("log")
plt.xscale("log")
plt.title("TTF v.s. DF plot(non-stopword removed)")
plt.xlabel("Rank")
plt.ylabel("TTF/DF value")
plt.plot(TTF.values(), label='TTF')
plt.plot(DF.values(), label='DF')
plt.legend()

```

We began with importing all the necessary packages such as json and nltk. We then started by opening every file in the yelp directory to save into an array of yelp_restaurants.

We loop through every document and perform the following:

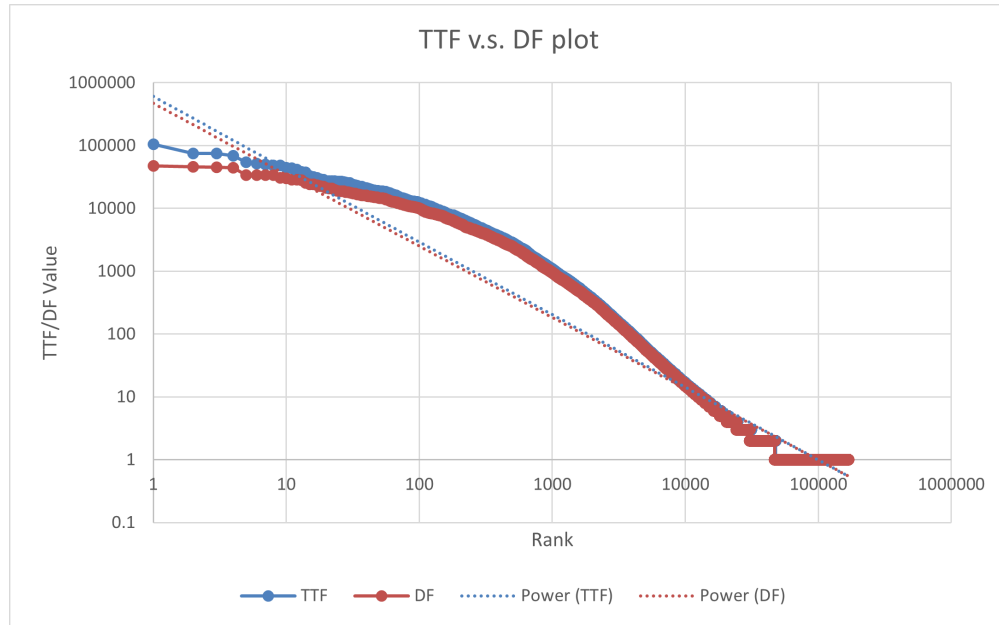
- i. normalize
- ii. tokenize
- iii. stopwords removal
- iv. write the new tokens as NormalizedContent for part 2
- v. count tokens as save it as BoWFreq for part 2
- vi. add to TTF for every word frequency
- vii. add 1 to DF for every word in frequency dictionary key

Once that is done, we sorted TTF and DF to be graphed as rank vs TTF/DF value. The next part shows the graphing done through matplotlib.

1.2. Two curves in log-log space

1.3. Answers

- From the resulting plot, can we find a strong linear relationship between the x and y axes in the log-log scale?
We do observe a linear relation between the x and y in the log-log scale.
- If so, what is the slope of this linear relationship?



TTF curve fit has a slope of -0.89 with a y-intercept of 90,998.2 and an x-intercept of 101,699.7.

DF curve fit has a slope of -0.88 with a y-intercept of 89,315.2 and an x-intercept of 101,407.8.

We observe that TTF fits Zipf's law better because common words tend to appear more often than others, and simply limiting them to count once per document would not show how often they occur but only that they would appear in almost every document, this then becomes a frequency observation between documents instead of the English language. Therefore, TTF fits Zipf's law better because it is a representation of a language in this use case instead of the representation of possible words in documents.

2. Building An Inverted Index

```
2.1. import time
    import re
    import string
    from elasticsearch import Elasticsearch
    es = Elasticsearch('http://localhost:9201')

    # Index every document by tokenized content
    for restaurant in yelp_restaurants:
        for document in restaurant['Reviews']:
            resp = es.index(index="yelp", id=document['ReviewID'], document={'Norma
            if resp['result'] != 'created' and resp['result'] != 'updated':
                print(resp['result'])

    queries = ["general chicken",
```

```

"fried chicken",
"BBQ sandwiches",
"mashed potatoes",
"Grilled Shrimp Salad",
"lamb Shank",
"Pepperoni pizza",
"brussel sprout salad",
"FRIENDLY STAFF",
"Grilled Cheese"]

# Perform same text processing as before
for i in range(len(queries)):
    #query normalization
    tokens = re.sub(r"\b\d+\b", "#", queries[i]
                    .translate(str.maketrans('', '', string.punctuation))
                    ).lower()

    # Tokenize by word
    tokens = nltk.word_tokenize(tokens)

    # # Stopword removal
    tokens = [word for word in tokens if not word in stop_words]

    # Token stemming
    queries[i] = [nltk.stem.PorterStemmer().stem(token) for token in tokens]

# Use the count method to obtain the count of docs matching query
now = time.time_ns() / (10 ** 9)
for query in [' '.join(query) for query in queries]:
    resp = es.count(index="yelp", query={"match": {"NormalizedContent":query}})
    print(f"Got %d Hits on '{query}':" % resp['count'])
print(f"Execution time: {time.time_ns()/(10**9) - now}s\n")

# Loop through every document and increment count if any query exist in the fre
now = time.time_ns() / (10 ** 9)
for query in queries:
    count = 0
    for restaurant in yelp_restaurants:
        for review in restaurant['Reviews']:
            if any( [review['BoWFrequency'].get(word, False) for word in query]
                    count += 1
            print(f"Got {count} Hits on '{' '.join(query)}'")
print(f"Execution time(BoW brute force): {time.time_ns()/(10**9) - now}s")

# print first searched doc and it's term_statistics
# ids=['SUTqIHsRs7_IaV5KJj41DA', 'FiGXnqZtZ09SSe5No2JyKQ', 'BmN2NU5JhkniSXmzLYy

```

```
# for i in range(len(queries)):
#     res = es.termvectors(index='yelp', id=ids[i], term_statistics=True, field
#     print(f"{queries[i]}:", res)
```

The above shows the code where we began with starting the elasticsearch connection on localhost on port 9201. We then go through every document to index the normalized content from part 1 into the es index known as 'yelp'. From here, we would process our query just like the document in part 1. Then, we are able to use the search method to obtain the number of hits of the term on the indexed document. By inserting a time.time_ns, we are able to track the time taken for the operation.

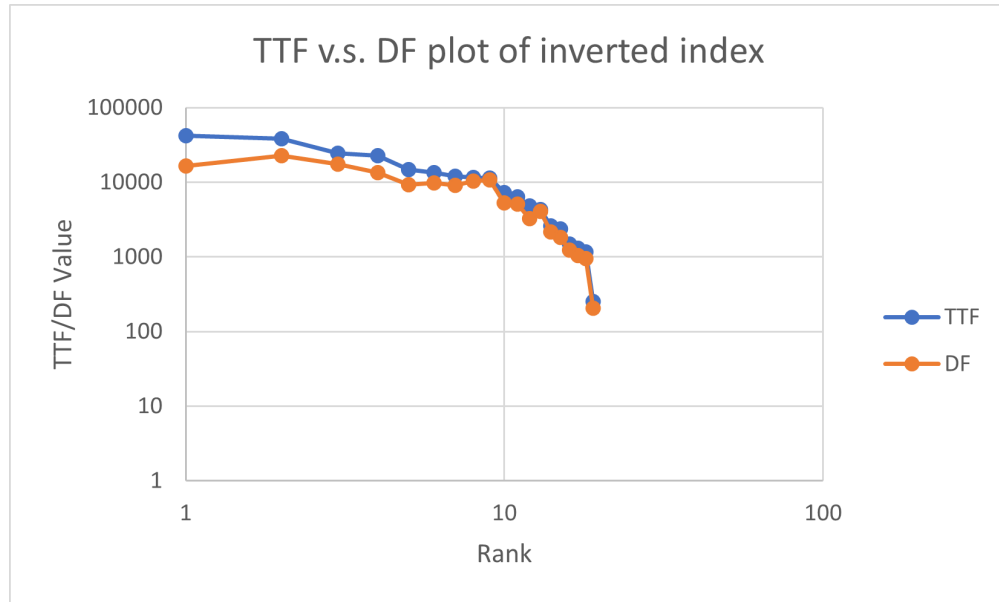
For the BoW brute force, We have saved a frequency table in part 1 which has the format of 'word': freq and this serves as our BoW implementation, so we searched through every document and got the query word on the document, the fetched freq is either 0 or a positive value, this is fed into the 'any' class so that it returns true if any query words exist in the document.

To obtain the TTF and DF from the elasticsearch, we used search instead of count just like before and saved the id of the first document that has the query. We then looked into the response for the target word which shows the TTF and DF as below:

```
'gener': {'doc_freq': 4085, 'ttf': 4358}
'fri': {'doc_freq': 22885, 'ttf': 38403}
'chicken': {'doc_freq': 13494, 'ttf': 22566}
'bbq': {'doc_freq': 3248, 'ttf': 4861}
'sandwich': {'doc_freq': 9330, 'ttf': 14709}
'mash': {'doc_freq': 2168, 'ttf': 2612}
'potato': {'doc_freq': 9081, 'ttf': 12086}
'grill': {'doc_freq': 5116, 'ttf': 6379}
'shrimp': {'doc_freq': 5335, 'ttf': 7331}
'lamb': {'doc_freq': 1828, 'ttf': 2376}
'shank': {'doc_freq': 205, 'ttf': 253}
'pepperoni': {'doc_freq': 1237, 'ttf': 1490}
'pizza': {'doc_freq': 16658, 'ttf': 42104}
'brussel': {'doc_freq': 958, 'ttf': 1168}
'sprout': {'doc_freq': 1054, 'ttf': 1301}
'salad': {'doc_freq': 9769, 'ttf': 13589}
'friendli': {'doc_freq': 10801, 'ttf': 11377}
'staff': {'doc_freq': 10380, 'ttf': 11589}
'chees': {'doc_freq': 17454, 'ttf': 24358}
```

2.2. Two curves in log-log space

2.3. Answers



- Running time and total number of returned documents by two retrieval models

```

Got 14595 Hits on 'gener chicken'
Got 25839 Hits on 'fri chicken'
Got 10727 Hits on 'bbq sandwich'
Got 7809 Hits on 'mash potato'
Got 14751 Hits on 'grill shrimp salad'
Got 1483 Hits on 'lamb shank'
Got 10762 Hits on 'pepperoni pizza'
Got 8403 Hits on 'brussel sprout salad'
Got 13971 Hits on 'friendly staff'
Got 16503 Hits on 'grill chees'
Execution time: 0.02199554443359375s

```

```

Got 14594 Hits on 'gener chicken'
Got 25839 Hits on 'fri chicken'
Got 10727 Hits on 'bbq sandwich'
Got 7808 Hits on 'mash potato'
Got 14752 Hits on 'grill shrimp salad'
Got 1483 Hits on 'lamb shank'
Got 10764 Hits on 'pepperoni pizza'
Got 8404 Hits on 'brussel sprout salad'
Got 13971 Hits on 'friendly staff'
Got 16503 Hits on 'grill chees'
Execution time(BoW brute force): 0.7786240577697754s

```

We observe that the inverted index is faster, the BoW implementation of part

1 is slow because it is unoptimized for bigger dictionary. To further optimize it, we should not keep a BoW in every dictionary but a general BoW that has the format of 'word': ['doc': freq] so we can query for existence in specific doc in $O(1)$ time instead of $O(n)$. We could also calculate the frequency of it in the doc by using sum.

- If you found these two methods gave you different number of matched documents, do you have any explanation about it?
They gave slightly different outputs, off by 1, this could be how elasticsearch normalizes their index and certain edge case have their words altered and uncounted.